

Panda Mock Robot Joint Replay 설계 문서

대상 환경: ROS 2 Humble, MoveIt 2, rosbag2, Panda mock components

목적:

1. 1차 목적: demo playback

2. 2차 목적: teaching 목적의 동작 재현

기록 소스: `/joint_states`

기록 방식: rosbag2 저장 후 후처리로 replay 데이터 생성

입력 주기: 50 Hz

제약:

- 기록 로봇과 replay 로봇 모두 실제 Panda가 아닌 mock component
- gripper 제외
- 저장 시점과 replay 시점의 joint 목록 동일
- 상태 기반 replay 고려
- 전체 trajectory 변환 방식 고려
- 전체 과정에 MoveIt 활용

1. 설계 목표

이 시스템의 목표는 `/joint_states`로부터 수집한 Panda arm의 joint 상태 이력을 저장하고, 이를 나중에 읽어 동일하거나 매우 유사한 관절 운동으로 재생하는 것이다.

단, `/joint_states`는 본질적으로 **상태 관측값(state observation)**이며, 제어 명령(command)이 아니다.

`sensor_msgs/JointState`는 joint 이름과 함께 position, velocity, effort를 담고, 하나의 메시지 안에서는 모든 joint 상태가 같은 시점에 기록되어야 하며, 각 배열은 같은 길이거나 비어 있어야 한다. 따라서 replay 설계는 “상태를 어떻게 제어 입력으로 변환할 것인가”를 핵심 문제로 다뤄야 한다. ([ROS Documentation](#))

이 요구사항에서는 `joint_states`를 그대로 다시 publish하는 방식보다, rosbag2에서 샘플을 읽어 후처리로 `trajectory_msgs/JointTrajectory` 또는 MoveIt의 `RobotTrajectory`로 재구성하고, 이를 `joint_trajectory_controller`에 전달하는 구조가 더 적합하다. `joint_trajectory_controller`는 joint-space trajectory 실행용 컨트롤러이며, 주어진 waypoint 사이를 시간 기준으로 보간하고, waypoint에는 position과 선택적으로 velocity/acceleration을 담을 수 있다. ([ROS Documentation](#))

2. 요구사항 해석과 핵심 결론

2.1 상태 기반 replay의 의미

상태 기반 replay는 “기록된 각 시점의 joint 상태를 목표 상태로 다시 따라가게 한다”는 접근이다. 하지만 `/joint_states` 는 관측값이지 실행 명령이 아니므로, 다음 두 방식 중 하나로 변환해야 한다.

1. 샘플-바이-샘플 재생

- 각 20 ms 샘플(50 Hz)을 순차 목표점으로 사용
- 매우 단순하지만 타이밍 지터와 추종 오차에 취약

2. 전체 trajectory 재구성 후 실행

- bag 전체를 읽어 time-indexed trajectory 생성
- 시작 시각 기준으로 상대시간을 부여
- 필요 시 velocity/acceleration 보강
- controller/action 기반으로 일괄 실행

이번 요구에서는 **demo playback + teaching 목적**이므로, 기본 설계는 **전체 trajectory 기반 replay**로 두고, 보조적으로 상태 기반 샘플 재생을 디버깅/실험 모드로만 제공하는 것이 타당하다.

2.2 MoveIt의 역할

MoveIt은 원래 **kinematic motion planning framework**이며, 경로(path) 자체에는 시간 정보가 없고, 후처리로 time parameterization을 추가해 velocity/acceleration이 있는 trajectory를 만든다. 또한 MoveIt의 TOTG(Time-Optimal Trajectory Generation)는 원래 waypoint와 약간 달라질 수 있는 샘플링을 수행할 수 있다. 따라서 “기록한 궤적을 최대한 그대로 재현”하려는 replay에서는 MoveIt을 **재계획(planning)** 용도가 아니라, **trajectory 검증, robot model 일치 확인, joint limit 적용, 필요 시 시간 파라미터 보강** 용도로 제한하는 것이 좋다. ([MoveIt](#))

즉, 본 시스템에서 MoveIt은 다음 역할이 적절하다.

- joint model / limits 검증
- trajectory 유효성 검사
- 필요 시 time parameterization
- RViz 시각화
- 실행 전 초기 상태/충돌 상태 확인

반대로, 기록된 demo를 다시 planning pipeline에 넣어 새 계획을 생성하는 방식은 피하는 것이 좋다. 그 경우 원래 데모와 다른 경로가 나올 수 있기 때문이다. 이 결론은 MoveIt의 시간 파라미터화와 TOTG의 waypoint 변형 가능성에서 직접 따라온다. ([MoveIt](#))

3. 권장 아키텍처

3.1 전체 구조

A. 기록 단계

- mock Panda 구동
- /joint_states 를 rosbag2로 기록
- 필요 시 /clock , /tf , /tf_static 도 함께 기록
- bag metadata 보존

B. 후처리 단계

- rosbag2를 읽어 /joint_states 추출
- joint name 정렬/검증
- 샘플 간 시간차 계산
- 이상치 제거 / 중복 시각 제거 / 누락 처리
- JointTrajectory 또는 RobotTrajectory 생성
- 필요 시 velocity/acceleration 보간 또는 재계산
- YAML/JSON/CSV/직렬화 메시지 등 replay용 산출물 생성

C. 실행 단계

- replay 시작 시 현재 로봇 상태 확인
- 시작 상태와 첫 trajectory point 차이 검사
- 필요 시 pre-roll 정렬 동작 수행
- joint_trajectory_controller action/topic으로 trajectory 전송
- 실행 결과 모니터링
- teaching용 분석 지표 산출

4. 가장 중요한 설계 판단

4.1 /joint_states 를 직접 replay하지 말 것

/joint_states 는 상태 토픽이다. 이를 다시 publish해도 일반적인 ros2_control/MoveIt 실행 경로에서는 로봇이 움직이지 않는다. 실제 움직임은 보통 controller command interface 또는 trajectory action을 통해 발생한다. joint_trajectory_controller 는 적어도 position feedback을 기대하며, trajectory waypoint를 받아 실행한다. (control.ros.org)

따라서 설계 원칙은 아래와 같다.

- 기록: /joint_states
- 재생 입력: /joint_trajectory_controller/joint_trajectory 또는 FollowJointTrajectory action
- 보조 정보: 필요 시 MoveIt의 RobotTrajectory

이 원칙은 구조를 훨씬 명확하게 한다.

즉, “상태를 기록한다”와 “명령으로 재생한다”를 분리해야 한다.

4.2 상태 기반 replay보다 trajectory 기반 replay를 주 경로로 둘 것

50 Hz는 20 ms 간격이다. 데모 playback 관점에서는 충분히 쓸 만한 샘플링이지만, 샘플마다 독립적으로 목표 위치를 쏘는 방식은 다음 문제가 있다.

- executor 지터
- DDS 지터
- controller update 주기와의 부정합
- 샘플 간 불연속성
- velocity/acceleration 정보 부재 또는 품질 저하
- teaching 분석 시 재현성 저하

반면 전체 trajectory로 만들면:

- 시작부터 끝까지 상대시간이 명확해짐
- controller가 내부 보간 수행
- tolerances 관리 가능
- 반복 재생의 일관성 향상

- teaching용 비교 분석이 쉬워짐

joint_trajectory_controller 는 point 간 시간 기반 보간을 수행하고 1-point trajectory도 수용하지만, playback 품질 관점에서는 의미 있는 다점 trajectory가 더 적합하다. ([ROS Documentation](#))

5. 상세 설계 사항

5.1 기록 단계 설계

5.1.1 기록 대상 토픽

최소:

- /joint_states

권장:

- /clock
- /tf
- /tf_static

mock 환경이라도 /clock 을 함께 기록해 두면 시뮬레이션 시간 기준 분석과 재생 검증이 편해진다. 특히 replay 시 bag timestamp와 message header stamp 중 어떤 시간을 기준으로 쓸지 비교할 수 있다.

5.1.2 rosbag2 QoS 고려

ROS 2에서는 DDS QoS 호환성이 recording/playback에 영향을 준다. rosbag2는 recording/playback 시 QoS를 적응적으로 맞추려 하지만, 경우에 따라 explicit override가 필요하다. 특히 호환성에 실질적으로 영향을 주는 것은 reliability와 durability이며, 필요하다면 --qos-profile-overrides-path 로 override를 지정할 수 있다. ([ROS Documentation](#))

설계 권장:

- 기록 시 /joint_states 의 실제 QoS 확인
- 재생용 토픽이나 분석 노드에서 동일/호환 QoS 사용
- mock component가 best_effort인지 reliable인지 명시
- bag playback을 직접 제어 입력으로 쓰지 않을 것이므로, **후처리 노드 입력 QoS**와 **검증용 subscriber QoS**를 명시적으로 관리

5.1.3 타임스탬프 기준

후처리 시 기준 시간은 두 후보가 있다.

- **bag receive timestamp**
- `JointState.header.stamp`

권장 원칙:

- 기본은 `header.stamp` 우선
- `header.stamp` 가 0이거나 불연속이면 bag timestamp 사용
- 내부 표준 시간축은 replay 파일에 별도 저장

이유:

- `JointState` 문서상 header는 해당 joint 상태가 기록된 시점을 뜻한다. ([ROS Documentation](#))

5.2 후처리 단계 설계

5.2.1 joint 순서 정규화

사용자 조건상 저장 시점과 replay 시점의 joint 목록은 동일하다. 그래도 후처리 단계에서 반드시 아래를 수행해야 한다.

- 기준 joint order를 명시적으로 정의
 - 예: `panda_joint1 ... panda_joint7`
- bag에서 읽은 각 `JointState.name` 을 기준 순서로 재배열
- name set이 정확히 일치하는지 검증
- 누락/중복/오탈자 발생 시 즉시 실패 처리

이 검증은 이후 모든 trajectory point의 semantic consistency를 보장한다.

5.2.2 샘플 유효성 검사

각 메시지에 대해 확인:

- `name.size == position.size`
- `velocity/effort` 존재 시 길이 일치
- 배열 길이 불일치 여부

- NaN/Inf 여부
- timestamp 역전 여부
- 동일 timestamp 중복 여부

JointState 는 배열 길이가 같거나 비어 있어야 한다. 이 규칙을 위반하면 replay용 point 생성은 중단하는 것이 안전하다. ([ROS Documentation](#))

5.2.3 downsampling / filtering

입력 주기가 50 Hz라면 demo playback에는 대체로 충분하다.

하지만 모든 샘플을 그대로 trajectory point로 쓰면 다음 상황이 생길 수 있다.

- 정지 구간이 불필요하게 매우 길어짐
- 미세 노이즈까지 모두 포함됨
- teaching 데이터로 사용할 때 품질 저하
- action message 크기 증가

권장:

- 기본 모드: 원본 50 Hz 유지
- teaching 모드: 아래 조건으로 압축 옵션 제공
 - joint 변화량이 임계값 이하이면 샘플 생략
 - 긴 정지 구간은 interval만 남기고 압축
 - Savitzky-Golay 또는 저차 smoothing 선택 옵션
- 단, demo playback 기본 모드에서는 원본 보존 우선

5.2.4 velocity / acceleration 처리

joint_trajectory_controller 의 waypoint에는 position만 넣을 수도 있고 velocity/acceleration을 선택적으로 넣을 수도 있다. controller는 시간 보간을 수행하지만, 내부적으로 acceleration에서 velocity, velocity에서 position을 자동 적분해 주는 구조는 아니다. 또한 하드웨어/상태 인터페이스 조합에 요구조건이 있다.

(control.ros.org)

권장 전략:

- **1차 구현:** positions + time_from_start 만 사용
- **2차 고도화:** 중심차분으로 velocity 추정
- **3차 고도화:** acceleration 추가 또는 MoveIt time parameterization 활용

이유:

- mock 환경 + demo playback에서는 position-only trajectory만으로도 충분히 구현 가능
- velocity를 부정확하게 넣는 것보다, controller가 point 간 보간하도록 두는 편이 더 안정적인 경우가 많음
- teaching 품질을 올릴 때만 velocity/acceleration 품질 관리 필요

5.3 trajectory 생성 설계

5.3.1 JointTrajectory 생성 규칙

각 bag 샘플을 다음으로 변환한다.

- joint_names : 기준 순서 고정
- points[i].positions : 해당 시점 관절값
- points[i].time_from_start : ($t_i - t_0$)

선택:

- velocities
- accelerations

추가 권장:

- 첫 점 이전에 $time_from_start = 0$ 의 명시적 시작점 삽입
- 필요 시 마지막 hold point 추가

5.3.2 시작 상태 정렬(pre-roll alignment)

기록된 첫 샘플 상태와 replay 시작 시 실제 mock robot 상태가 다를 수 있다.

이 경우 바로 trajectory를 실행하면 첫 구간에서 급격한 점프가 생긴다.

권장:

- replay 전 현재 joint state를 읽는다
- 첫 point와의 차이가 threshold 이하이면 바로 시작
- threshold 초과이면 **정렬용 short trajectory**를 먼저 실행
- 정렬이 끝난 뒤 원래 demo trajectory를 실행

이 단계는 데모 시연 안정성에 매우 중요하다.

5.3.3 시간축 보존 정책

replay의 목적이 “같은 행동”이므로 시간축 정책을 명확히 해야 한다.

권장 기본:

- 원본 시간 간격 유지

옵션:

- speed_scale
 - 0.5x, 1.0x, 2.0x
- teaching 모드에서는 느리게 재생 허용

Movelt은 runtime에서 velocity/acceleration scaling factor를 통해 속도를 조정할 수 있으며, 파일 기반 joint limit 설정도 사용한다. 다만 replay 정확도가 목표라면, 기본 모드는 원본 time axis 보존이 우선이다. ([Movelt](#))

5.3.4 Movelt time parameterization 사용 조건

Movelt의 time parameterization은 path에 속도/가속도와 timestamp를 부여하는 데 유용하지만, TOTG는 결과 waypoint가 원래 궤적에서 약간 벗어날 수 있다. ([Movelt](#))

따라서 원칙:

- 이미 50 Hz state sequence와 상대시간이 존재하면, **그 자체를 trajectory timing으로 사용**
- Movelt time parameterization은 아래 경우에만 선택 사용
 - velocity/acceleration이 꼭 필요한 경우
 - 샘플 간 시간 이상치가 큰 경우
 - 속도 상한 초과가 발생해 limit-compliant trajectory가 필요한 경우

즉, Movelt time parameterization은 **기본값이 아니라 보정 옵션**으로 두는 것이 좋다.

5.4 실행 단계 설계

5.4.1 controller 인터페이스

joint_trajectory_controller 는 joint-space trajectory 실행용이며, point 간 시간 보간을 수행한다. even one-point trajectory도 허용되지만, 본 설계에서는 multi-point trajectory 실행을 기준으로 한다. ([ROS Documentation](#))

권장:

- FollowJointTrajectory action 기반 실행
- 상태 모니터링
- tolerances / goal_time 설정
- action preemption 정책 고려

문서상 한 시점에 하나의 action goal만 active할 수 있으며, 새로운 goal이 들어오면 기존 goal 처리 규칙이 적용된다. 따라서 “스트리밍처럼 자주 새 trajectory를 덮어쓰기”보다, **한 번에 완성된 trajectory를 보내는 구조**가 더 안정적이다. (control.ros.org)

5.4.2 controller 설정

문서 예시에서도 position command, position/velocity state, state publish rate 50Hz, allow_partial_joints_goal: false 같은 구성이 제시된다. joint 목록이 동일하다는 사용자 조건과도 잘 맞는다. (control.ros.org)

권장 설정 방향:

- allow_partial_joints_goal = false
- joints 순서 고정
- Panda arm 7축 전체 포함
- position command interface 기준
- state interface는 최소 position, 가능하면 velocity 포함
- tolerances는 데모용/teaching용 두 프로파일 분리

6. 반드시 주의할 점

6.1 “동일 행동”의 정의를 명확히 해야 함

mock component 환경에서는 실제 하드웨어 동특성, 마찰, controller tuning, transmission 오차가 없거나 단 순화되어 있을 수 있다. 그래서 “동일 행동”은 보통 다음 중 하나로 정의해야 한다.

- 관절 궤적 유사성
- 시간축까지 포함한 관절 상태 일치도
- 엔드이펙터 경로 유사성
- 시연 관찰상 동일성

본 프로젝트에서는 현실적으로 아래 정의가 적합하다.

- 1차 demo playback: 관절 궤적과 시간적 인상이 충분히 유사
- 2차 teaching: 각 관절 시계열 비교가 가능할 정도의 재현성 확보

6.2 bag에 기록된 노이즈를 그대로 “정답”으로 취급하지 말 것

특히 teaching 목적에서는 사람 조작이나 상위 제어기 미세 진동이 그대로 남을 수 있다.
그래서 저장 원본은 보존하되, replay용 산출물은 아래 2종으로 분리하는 것이 좋다.

- raw trajectory
- cleaned trajectory

6.3 첫 점프 문제

가장 흔한 실패 원인이다.

- 현재 로봇 상태 ≠ 데모 첫 상태
- trajectory 첫 점이 즉시 도달 목표가 됨
- 순간적인 큰 오차 발생

반드시 pre-roll alignment를 넣어야 한다.

6.4 시간 역전 / 중복 timestamp

bag 후처리에서 반드시 제거해야 한다.

특히 replay file 생성 전 다음을 강제:

- strictly nondecreasing time
- 중복 시각 merge or drop
- 최소 dt 보장

6.5 MoveIt 재계획 남용 금지

기록된 데모를 재생하는 시스템인데, planning request로 다시 풀어버리면 “비슷하지만 다른 motion”이 된다.
MoveIt은 검증/시각화/한계 체크/보정으로 쓰고, 원본 demo를 planning 문제로 다시 풀지 않는 것이 핵심이다. ([MoveIt](#))

6.6 joint limit와 wrap-around

연속 joint가 있는 경우 angle unwrap 문제가 생길 수 있다. `joint_trajectory_controller` 는 continuous joint handling을 지원한다. Panda에서는 보통 크게 문제 되지 않더라도, 후처리에서 각도 discontinuity를 검사하는 것이 좋다. (control.ros.org)

7. 권장 데이터 포맷

rosbag2 원본 외에, 후처리 산출물로 아래 포맷 중 하나를 권장한다.

7.1 권장 1안: YAML/JSON 메타 + trajectory binary/message

저장 항목:

- robot model name
- joint_names
- source bag path
- source topic
- recorded start/end time
- replay speed scale default
- sampling stats
- trajectory points
 - time_from_start
 - positions
 - optional velocities
 - optional accelerations

장점:

- 재현성과 디버깅 편의성 좋음

7.2 권장 2안: CSV + metadata

CSV:

- t, q1, q2, ... q7, dq1, ...

장점:

- teaching 분석 쉬움
단점:
- 실행 직결성은 다소 낮음

실무적으로는 **원본 rosbag2 + replay용 JSON/YAML + 분석용 CSV** 3종 보관이 가장 좋다.

8. 소프트웨어 모듈 분리 권장안

8.1 panda_demo_recorder

역할:

- rosbag2 record wrapper
- 기록 세션 메타데이터 관리
- 파일명, 태그, 시나리오 이름 관리

8.2 panda_demo_postprocessor

역할:

- bag 읽기
- /joint_states 파싱
- joint order 정렬
- timestamp 정제
- trajectory 생성
- raw / cleaned / teaching용 버전 산출

8.3 panda_demo_replayer

역할:

- 현재 상태 확인
- pre-roll alignment
- controller/action으로 trajectory 전송
- 결과 로깅

8.4 panda_demo_validator

역할:

- 원본 vs 재생 궤적 비교
- RMSE, max error, timing lag
- teaching용 리포트 생성

8.5 panda_moveit_bridge

역할:

- MoveIt robot model 로딩
- joint limits 검증
- optional time parameterization
- RViz visualization

9. 권장 개발 절차

아래 절차대로 가면 실패 가능성이 가장 낮다.

1단계. mock Panda 기본 실행 환경 정리

- Panda mock component + ros2_control + MoveIt 실행
- /joint_states 정상 발행 확인
- controller 이름과 command/state interfaces 확정
- joint_trajectory_controller 단일 trajectory 실행 확인

완료 기준:

- 수동 생성한 간단한 JointTrajectory 한 개를 정상 실행 가능

2단계. rosbag2 기록 파이프라인 구축

- /joint_states 기록
- 필요 시 /clock 동시 기록
- QoS 확인 및 override 필요 여부 검토

- 10~20초 demo 수집

완료 기준:

- bag에서 /joint_states 를 안정적으로 읽을 수 있음

3단계. bag 파서/후처리기 개발

- rosbag2 reader 구현
- JointState 파싱
- joint order 정규화
- timestamp 정제
- 메시지 유효성 검사
- CSV dump 생성

완료 기준:

- bag → 정렬된 시계열 데이터 변환 가능

4단계. raw trajectory 생성

- 각 샘플을 JointTrajectoryPoint 로 변환
- time_from_start 계산
- position-only trajectory 생성
- replay 파일 저장

완료 기준:

- bag → JointTrajectory 변환 가능

5단계. replay 노드 구현

- 현재 joint 상태 읽기
- 시작점 오차 검사
- 필요 시 alignment trajectory 실행
- 본 trajectory action 전송
- 완료 결과 수집

완료 기준:

- 단일 demo를 처음부터 끝까지 재생 가능

6단계. MoveIt 연동

- robot model로 joint names/limits 검증
- RViz 시각화
- optional time parameterization 연결
- limit 초과 및 부적합 trajectory 검출

완료 기준:

- replay 전 validation 단계가 자동 수행됨

7단계. 정량 평가 도구 개발

- 원본 vs replay 결과 비교
- joint별 RMSE
- max absolute error
- 시간 지연 분석
- teaching용 그래프 출력

완료 기준:

- demo playback 품질을 수치로 판단 가능

8단계. teaching 모드 추가

- cleaned trajectory 생성
- downsampling / smoothing 옵션
- speed scaling
- 구간 반복 replay
- 특정 구간 crop 기능

완료 기준:

- 시연용과 교육용 산출물을 분리 운영 가능

10. 테스트 전략

10.1 기능 테스트

- bag 기록 성공
- 후처리 성공
- trajectory 생성 성공
- replay 성공
- 동일 joint set 검증 성공

10.2 경계 테스트

- 빈 bag
- 일부 message 길이 불일치
- timestamp 중복/역전
- 첫 상태 불일치
- velocity 미포함
- 정지 구간 장시간 포함

10.3 품질 테스트

- 원본/재생 joint RMSE
- replay duration 오차
- point skip 여부
- 반복 replay 재현성

10.4 교육용 테스트

- speed scaling 0.5x, 1.0x, 2.0x
- 특정 구간 잘라서 replay
- raw vs cleaned 차이 비교

11. 최종 권장안

현재 조건에서는 아래 구조가 가장 적절합니다.

권장 구현 선택

- 기록: rosbag2로 /joint_states
- 후처리: bag → 정규화된 JointTrajectory
- 재생: joint_trajectory_controller action 기반
- **MovelT 역할**: 검증, 시각화, limit 체크, 선택적 time parameterization
- 기본 trajectory 데이터: positions + time_from_start
- 옵션 데이터: velocity/acceleration, smoothing, speed scaling
- 시작 전 처리: 반드시 pre-roll alignment 수행

왜 이 구성이 좋은가

- 상태 토픽과 제어 토픽을 명확히 분리
- demo playback에 가장 안정적
- teaching 확장 용이
- mock 환경에서 빠르게 검증 가능
- 이후 실제 하드웨어 이전 시에도 구조를 거의 유지 가능

12. 구현 우선순위

가장 먼저 만들 것:

1. bag → joint 시계열 추출기
2. 시계열 → JointTrajectory 변환기
3. replay 실행기
4. 시작 상태 alignment
5. 원본/재생 비교기

나중에 붙일 것:

1. MovelT 기반 validation
2. smoothing/downsampling
3. teaching용 구간 편집

- 4. 속도 배율 재생
- 5. trajectory 품질 지표 리포트

13. 한 줄 결론

이 프로젝트는 `/joint_states` 를 기록하고, 이를 후처리로 `JointTrajectory` 로 변환해 `controller`에 보내는 구조가 정석입니다.

Movel은 재계획 도구가 아니라 검증/보정 도구로 쓰는 것이 핵심이고, 가장 중요한 실전 이슈는 **QoS**, **timestamp** 기준, **joint** 순서 정규화, 시작 상태 정렬, 원본 시간축 보존입니다. ([ROS Documentation](#))

원하시면 다음 단계로 이어서 이 설계를 바탕으로 **ROS 2 Python(ament_python)** 기준 패키지 구조, 노드 구성, 메시지 흐름, 클래스 설계, 그리고 MVP 구현 순서까지 구체화해 드리겠습니다.